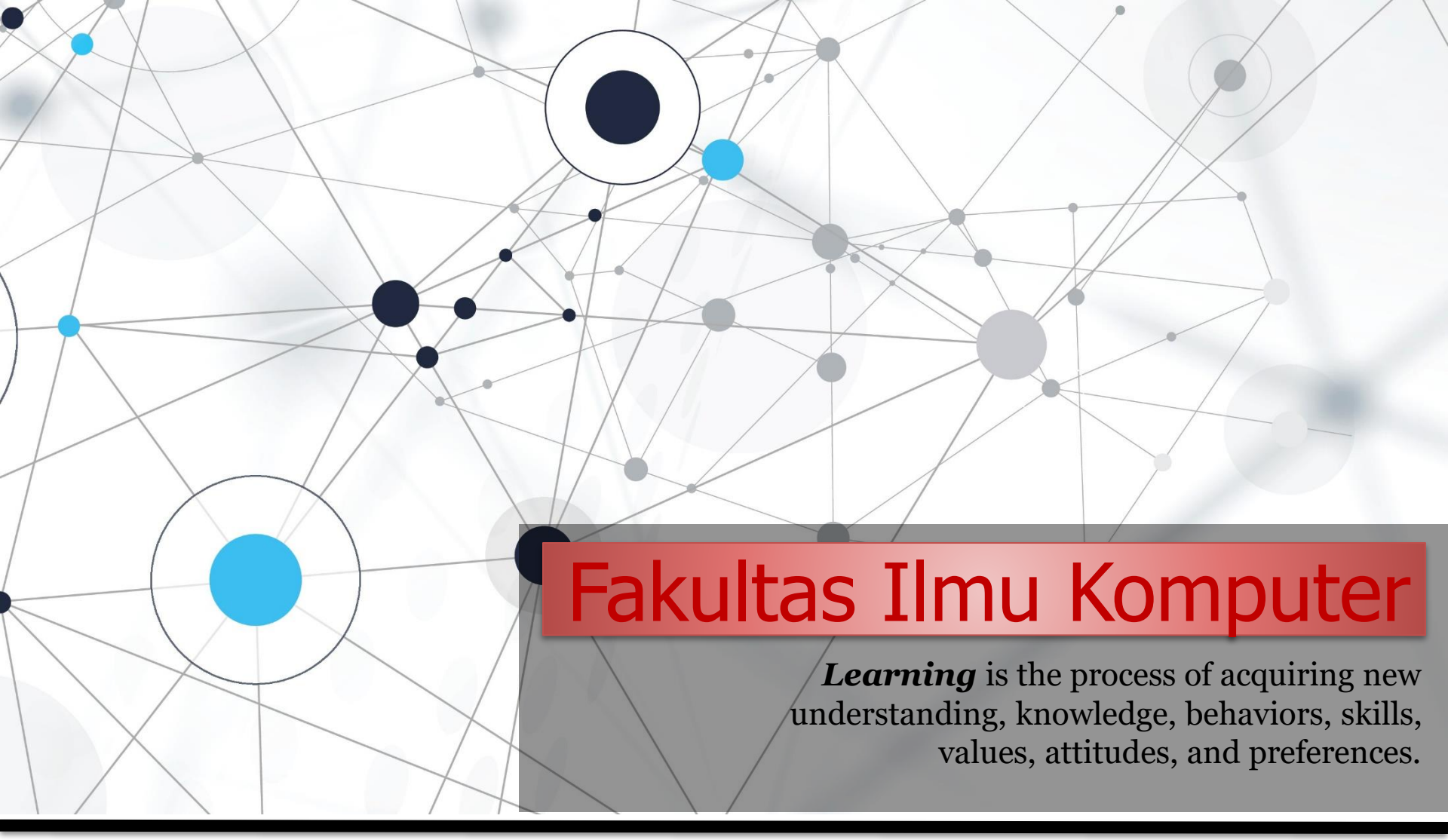
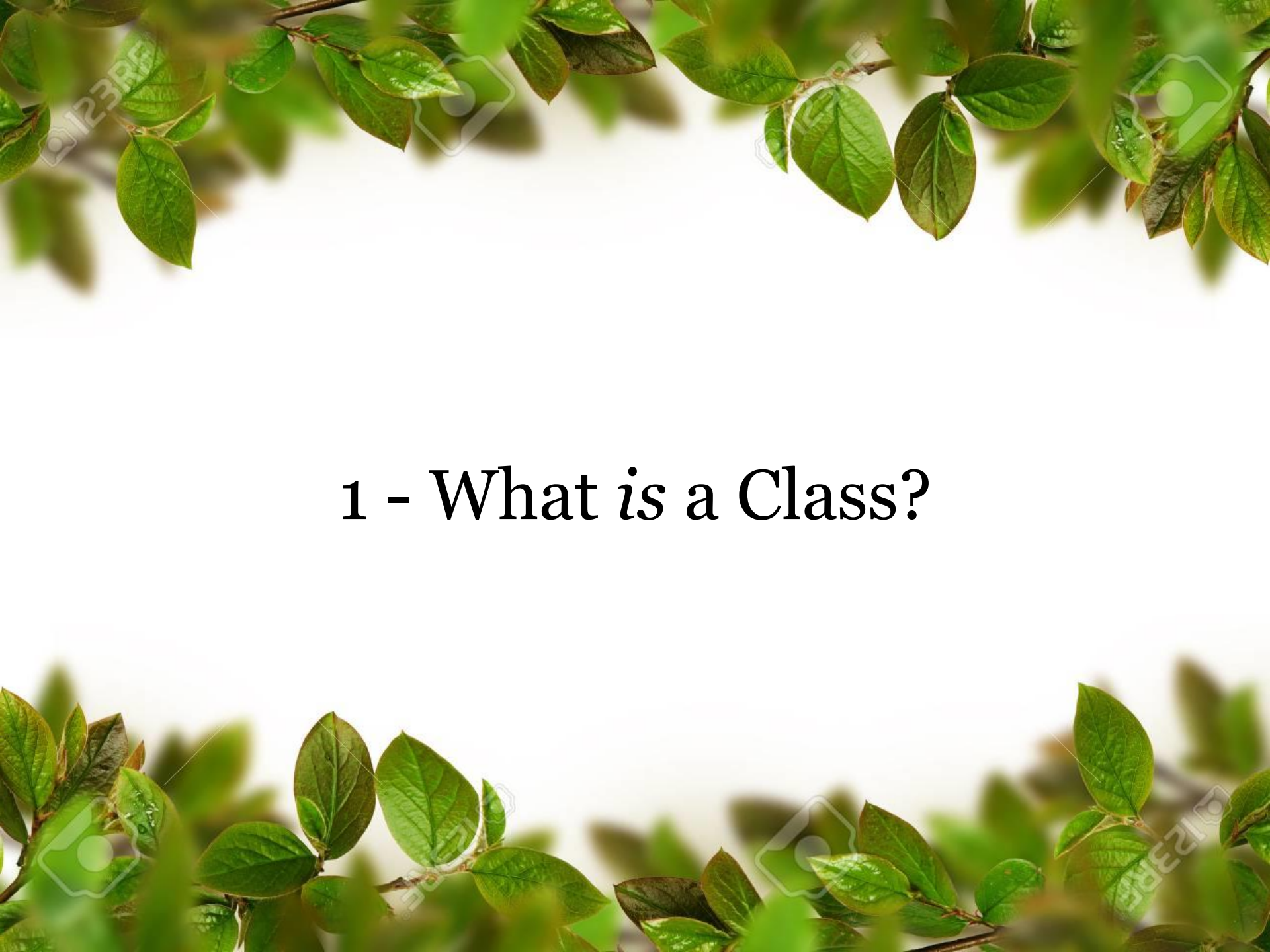


OOP in Python – Classes/Objects

(*Parent Classes, Child Classes, Instance Attributes*)

Welcome to *the* Object Oriented Programming *course*

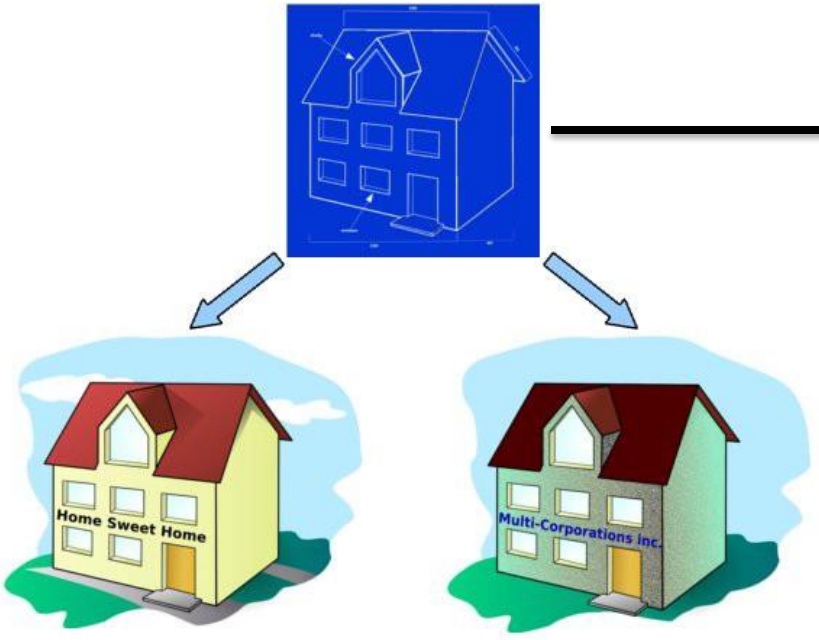


1 - What *is* a Class?

What *is* a Class?

1) A **user-defined prototype (blueprint, design)** for an object that defines a set of attributes that characterize any object of the class. The **attributes** are **data members (class variables and instance variables)** and methods, accessed via dot notation.

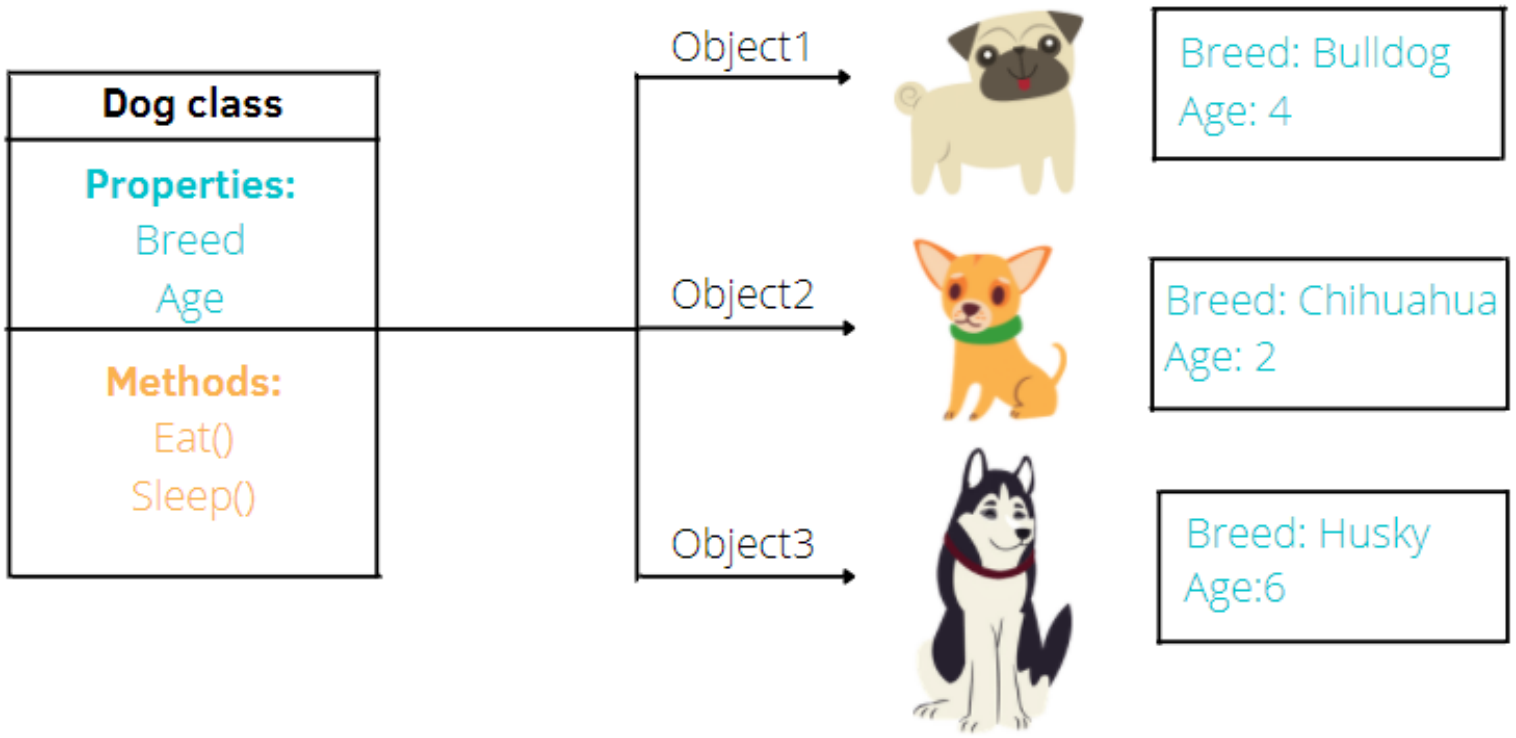
2) A **Class in Python** is a **logical grouping of data** and **functions**. It gives the freedom to create **data structures** that contains arbitrary (*yg berubah-ubah, dynamic*) content and hence easily accessible.



Class, Prototype, blueprint, design

Class atau kelas-kelas adalah *prototipe* untuk wadah menghimpun data dan kebergunaan yang kemudian menghasilkan objek. Setiap class baru akan menghasilkan objek baru yang kemudian bisa dibuat instance dengan memiliki atribut yang ada.

What *is* a Class?

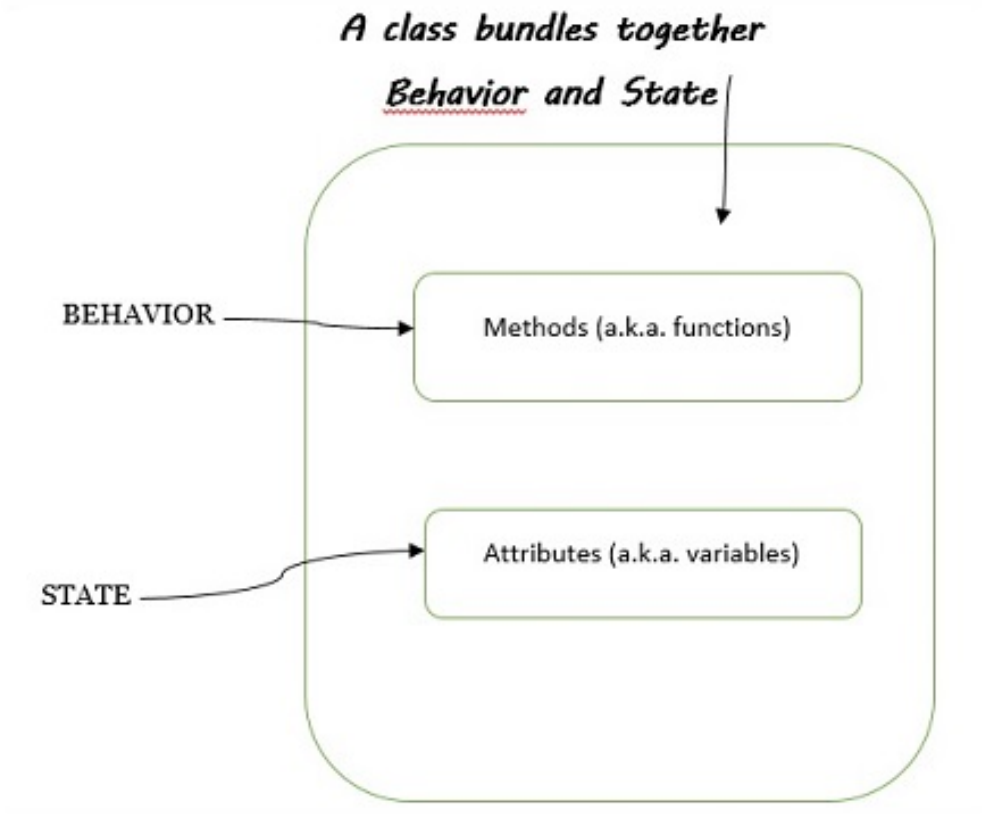


What *is* instance in Python?

Dalam pemrograman berorientasi objek (OOP) di Python, istilah "instance" merujuk kepada **OBJECT** yang dibuat dari suatu **CLASS**.

Class Bundles : Behavior and State

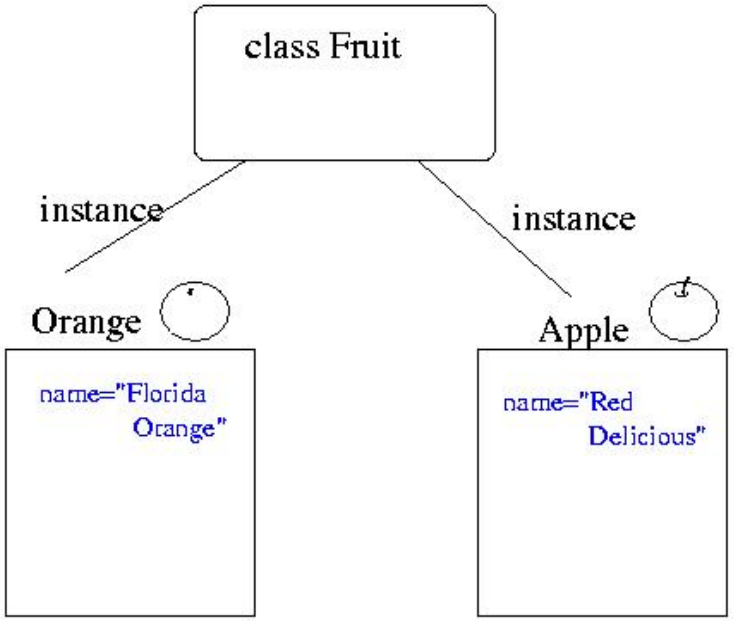
- The word **behavior** is identical to **function** – it is a piece of code that does something (*or implements a behavior*)
- The word **state** is identical to **variables** – it is a place to store values within a class.

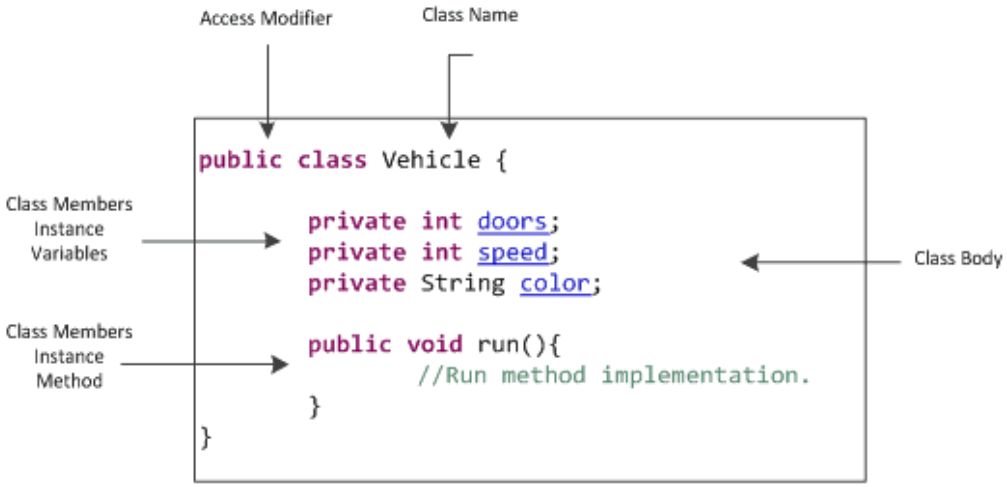


When we assert/define/declare a class behavior and state together, it means that a class packages functions and variables.

What *is* instance *in* Python?

In the Python programming language, an **instance of a class** is also called an **object**. The call will comprise **both data members and methods** and will be accessed by an object of that class.





A class is declared by use of the class keyword. The class body is enclosed between curly braces { and }. The data or variables, defined within a class are called **instance variables**. The code is contained within methods. Collectively, the methods and variables defined within a class are called **members of the class**.

Overview *of* OOP Terminology

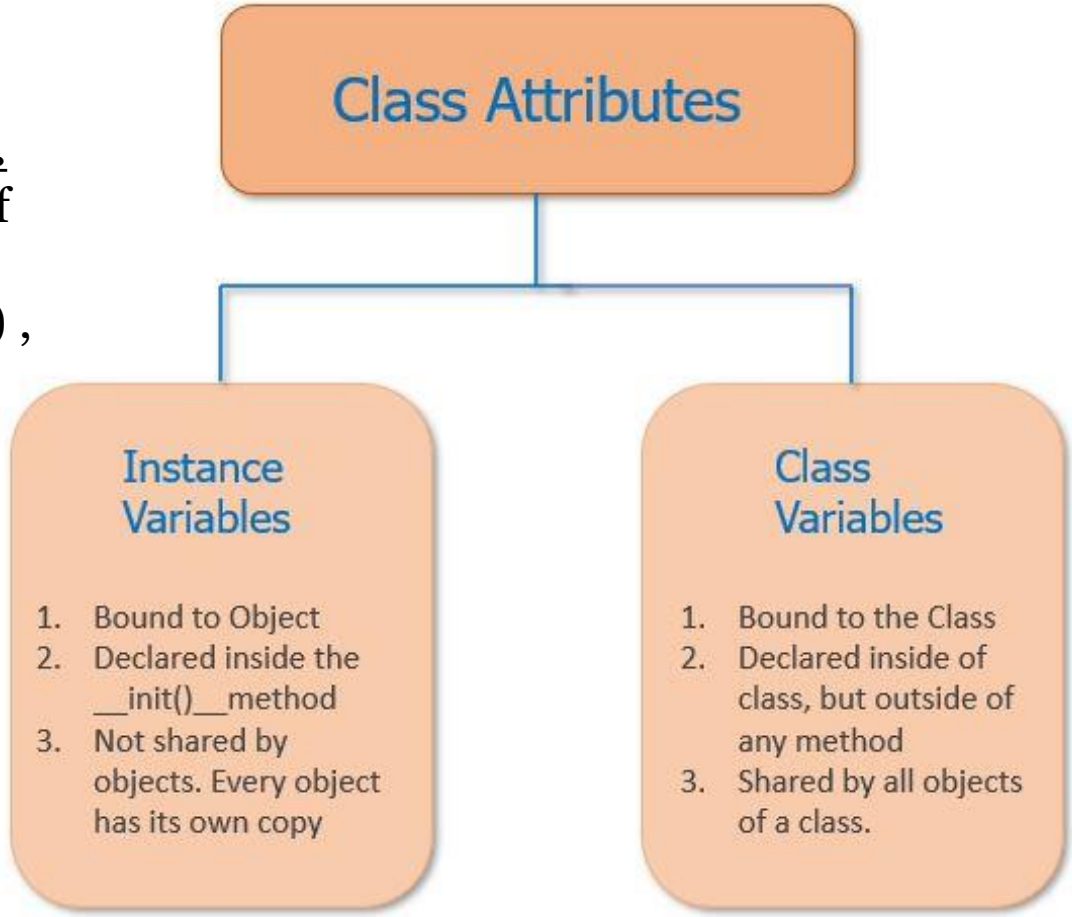
Class is a **user-defined prototype for an object** that defines a set of attributes that characterize any object of the class. The attributes are data members (*class variables and instance variables*) and methods, accessed via dot notation.

- Class variable is a variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- Data member is a class variable or instance variable that holds data associated with a class and its objects.
- Inheritance is the transfer of the characteristics of a class to other classes that are derived from it.
- Instance is an individual object of a certain class. An object obj that belongs to a class Circle, for example, is an instance of the class Circle.
- Instantiation is the creation of an instance of a class.

- Instance variable is a variable that is defined inside a method and belongs only to the current instance of a class.
- Object is a unique instance of a data structure that's defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- Method is a special kind of function that is defined in a class definition.
- Function overloading is the assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- Operator overloading is the assignment of more than one function to a particular operator.

Class Attributes

A class attribute is a Python variable that belongs to a class rather than a particular object. It's shared between all the objects of this class and is defined outside the constructor function, ___init___(*self*) , of the class.



Instance variable is a variable that is defined **inside a method** and belongs only to the current instance of a class.

Classes have methods & attributes

In Python, **creating a method defines a class behavior**. The word method is the OOP name given to a function that is defined within a class.

1) **Class functions** is synonym for **methods**.

2) **Class variables** is synonym for name **attributes**.

3) **Class** is a blueprint/template/design for an instance with exact **behavior**.

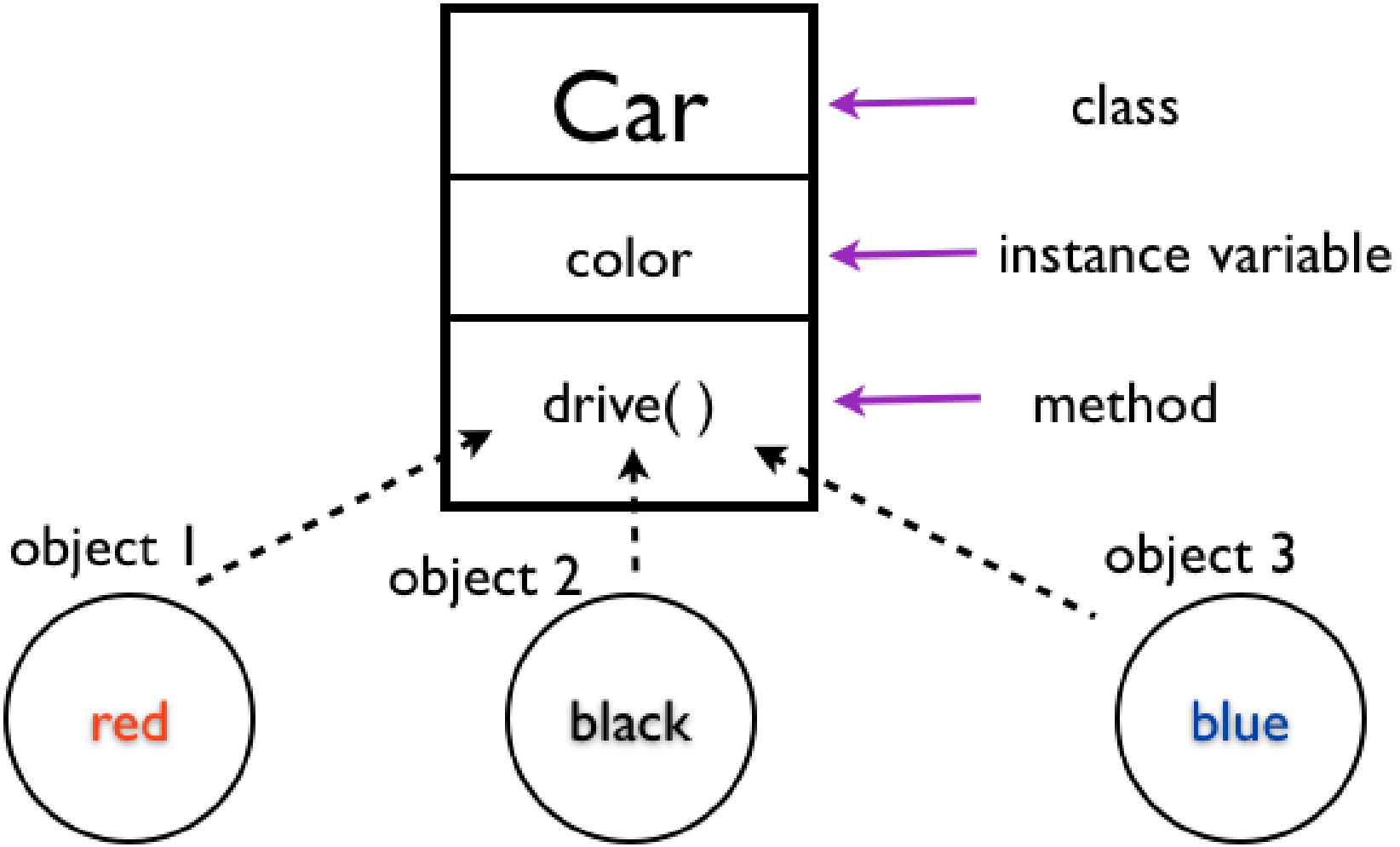
4) **Object** is one of the instances of the class, perform functionality defined in the class.

5) **Type** is indicates the class the instance belongs to.

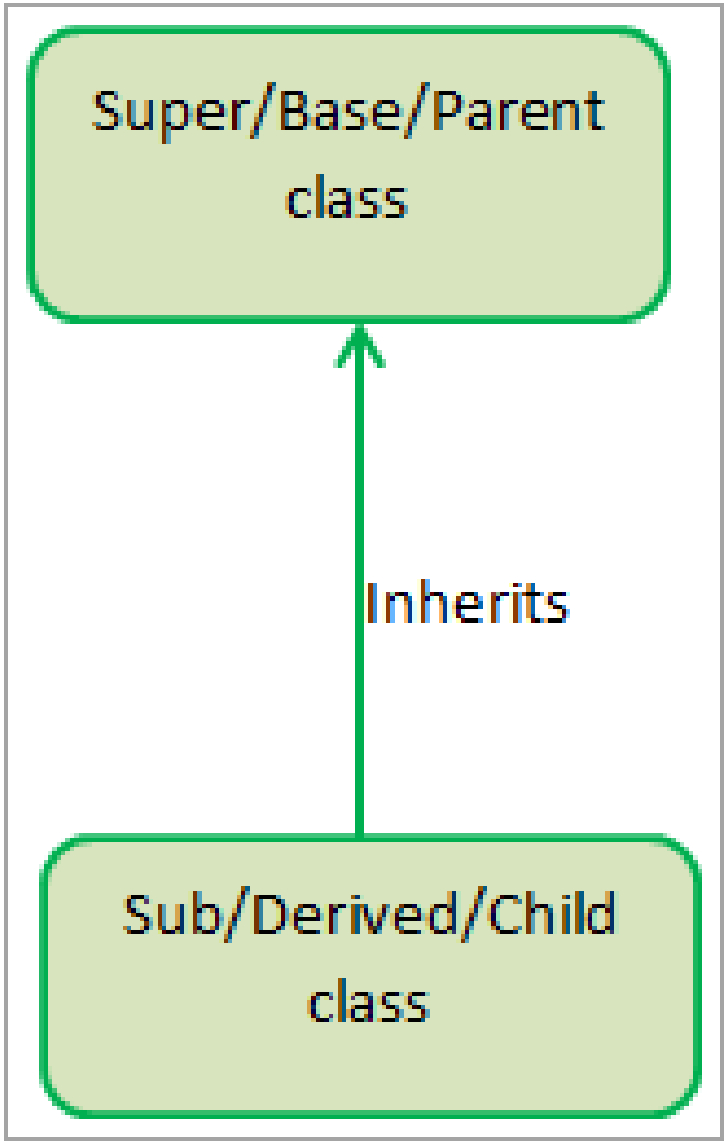
6) **Attribute** is any object value: object.attribute.

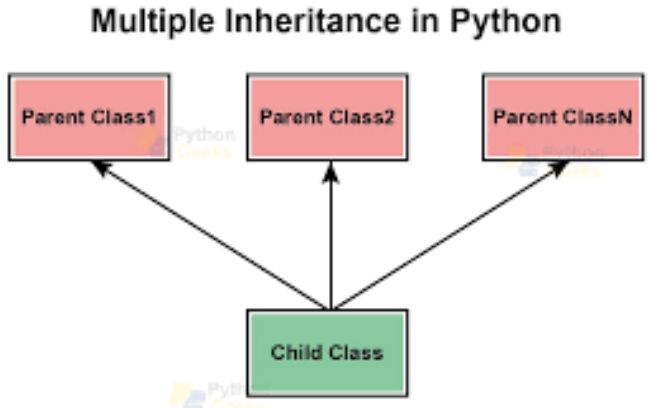
7) **Method** is a "*callable attribute*" defined in the class.

Multiple Objects Share Single Method



Parent Class & Child Class *in* Python





Python Inheritance:
Inheritance allows us to define a class that inherits all the methods and properties from another class.

1) **Parent class** is the class being inherited from, also called **base class**.

2) **Child class** is the class that inherits from another class, also called **derived class**.

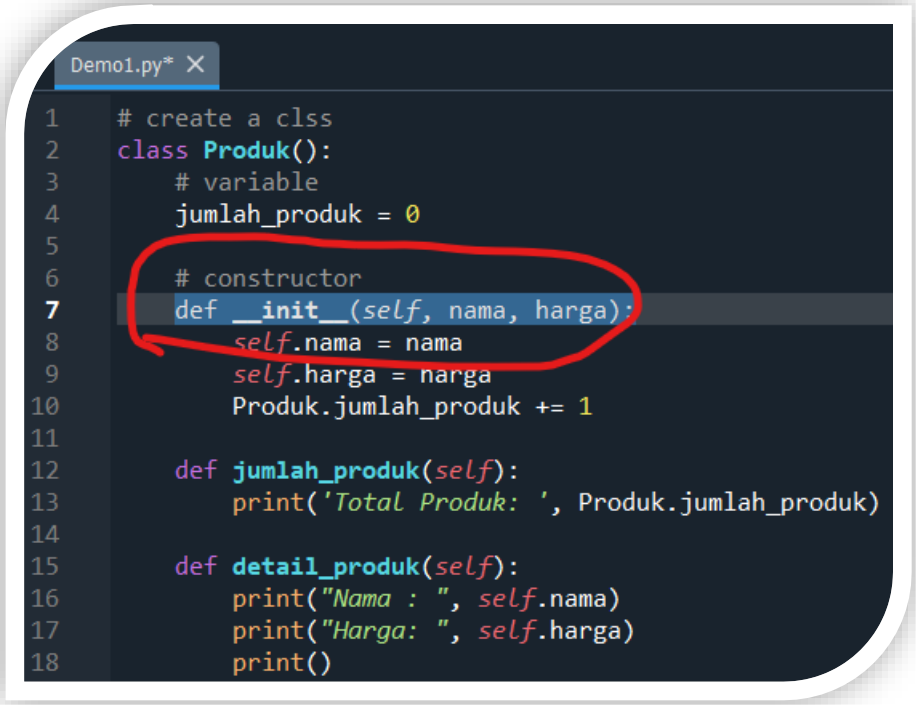
2 – How *to* Create Classes?

Creating Classes

Cara membuat sebuah *Class* menggunakan bahasa pemrograman Python sebenarnya cukup mudah, hanya dengan menggunakan kata kunci "**class**" diikuti dengan ***nama kelas*** dan diakhiri dengan titik dua (:). **Umumnya nama dari sebuah *class* OOP akan diawali dengan huruf *capital*.**

```
1  class NamaKelas():
2      """ membuat class """
```

"*def __init__(self)*" artinya?

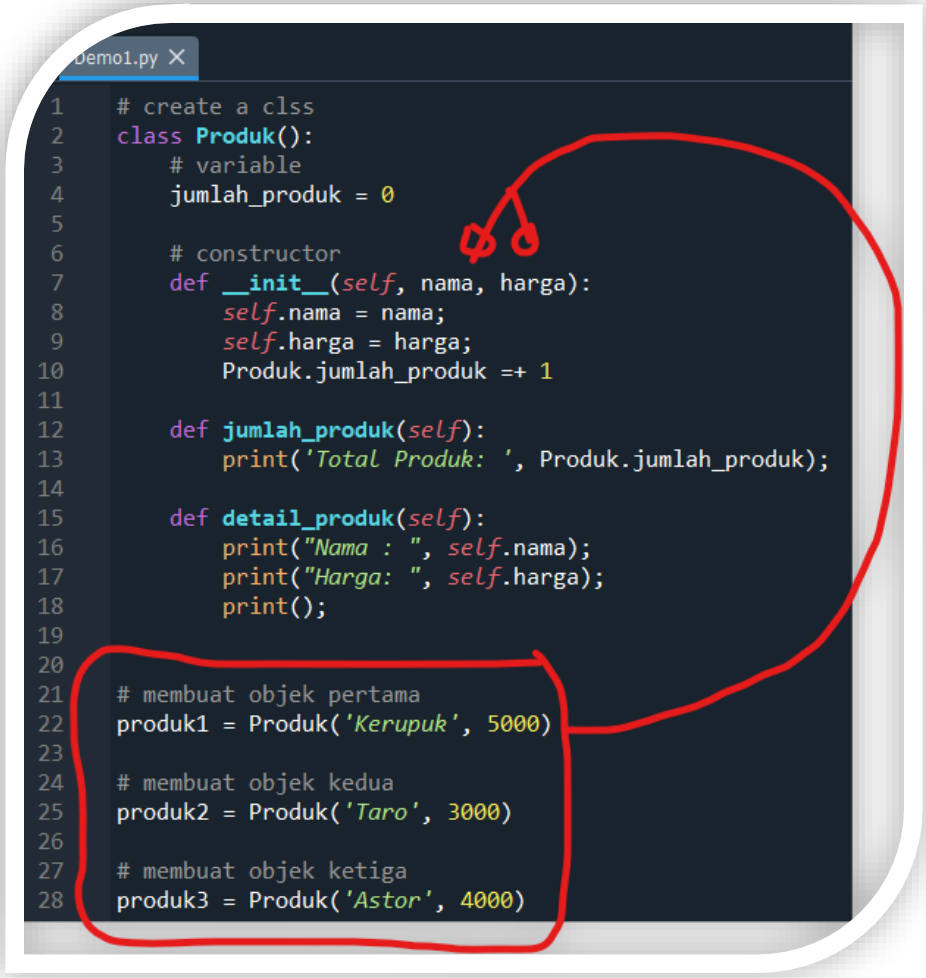


def __init__(self) adalah sebuah **constructor function**. Function ini akan dipanggil ketika object baru pertama kali di buat.

Jadi, *constructor* umumnya digunakan untuk menginisialisasi atribut/variable objek.

Def __init__(*self*) adalah metode **constructor** yang menginisialkan pembentukan objek dari kelas.

Instance Object



Untuk membuat **objek instance**, caranya dengan memanggil class kemudian isi argumen sesuai dengan pada **fungsi ___init___()** yang sudah kita definisikan sebelumnya.

Mengakses Properties Objek ?

Demo1.py*

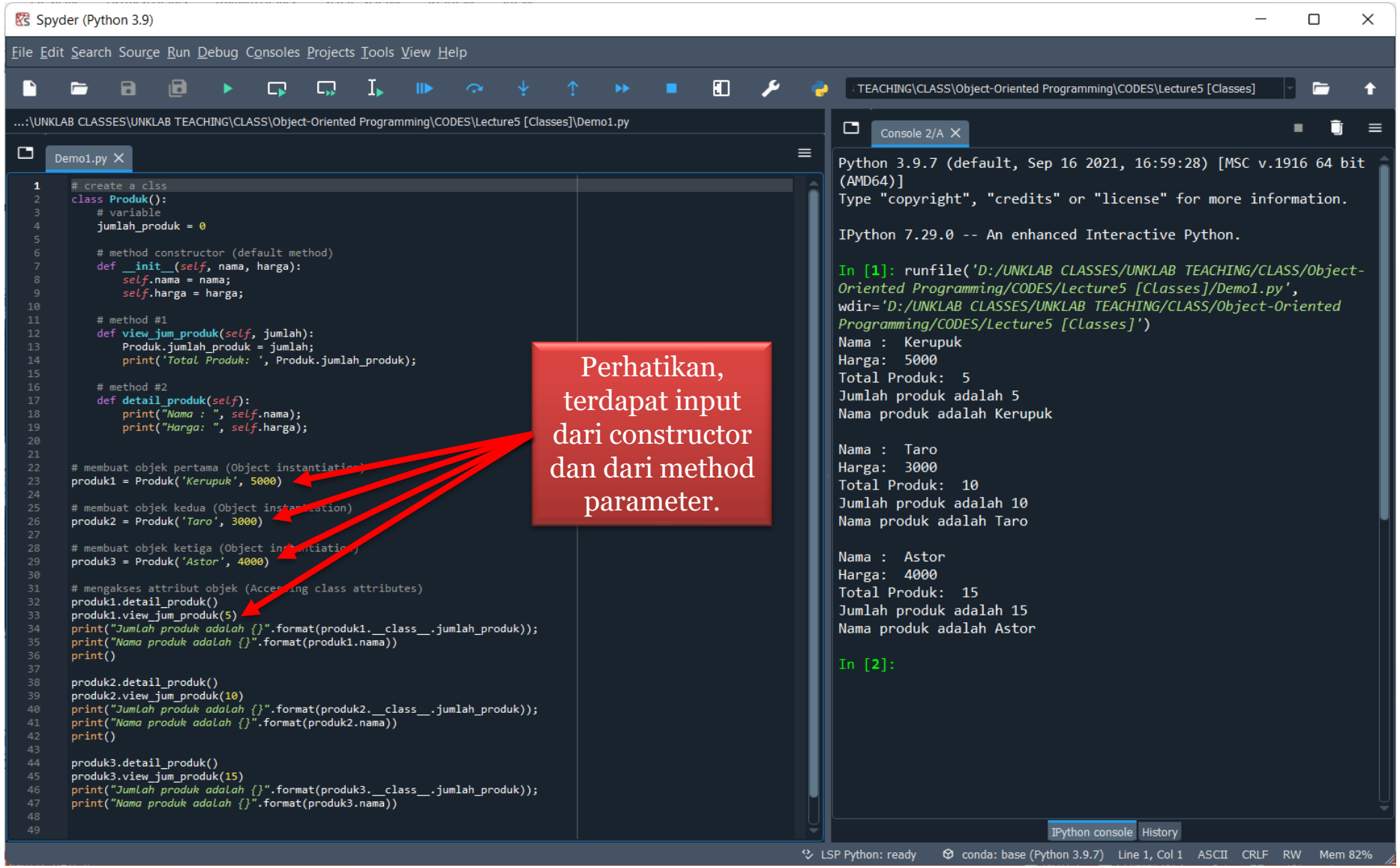
```python
# create a clss
class Produk():
    # variable
    jumlah_produk = 0

    # constructor
    def __init__(self, nama, harga):
        self.nama = nama;
        self.harga = harga;

    def view_jum_produk(self, jumlah):
        Produk.jumlah_produk = jumlah;
        print('Total Produk: ', Produk.jumlah_produk);

    def detail_produk(self):
        print("Nama : ", self.nama);
        print("Harga: ", self.harga);
        print();


# membuat objek pertama
produk1 = Produk('Kerupuk', 5000)

# membuat objek kedua
produk2 = Produk('Taro', 3000)

# membuat objek ketiga
produk3 = Produk('Astor', 4000)

# mengakses attribut objek
produk1.detail_produk()
produk2.detail_produk()
produk3.detail_produk()

produk1.view_jum_produk(5)
produk2.view_jum_produk(10)
produk3.view_jum_produk(15)
```

Untuk dapat mengakses properties dari objek (attribute/variable dan juga method) dengan cara variabel object (.) variabel kelas.

Menurut anda, apakah kita dapat membuat *class* di Python tanpa memiliki konstruktor (*constructor*) atau metode __init__ ???

Exercise *for* Students

Exercise #1 (Class & Objects)



Modifikasi source code pada file Demo1.py:

1. Dalam **class Produk**, tambahkan 2 *variable* (***tanggal_kedaluwarsa*** dan ***negara_asal***) dengan type String.
   tanggal_kedaluwarsa (expired date)
   negara_asal (made in Indonesia)

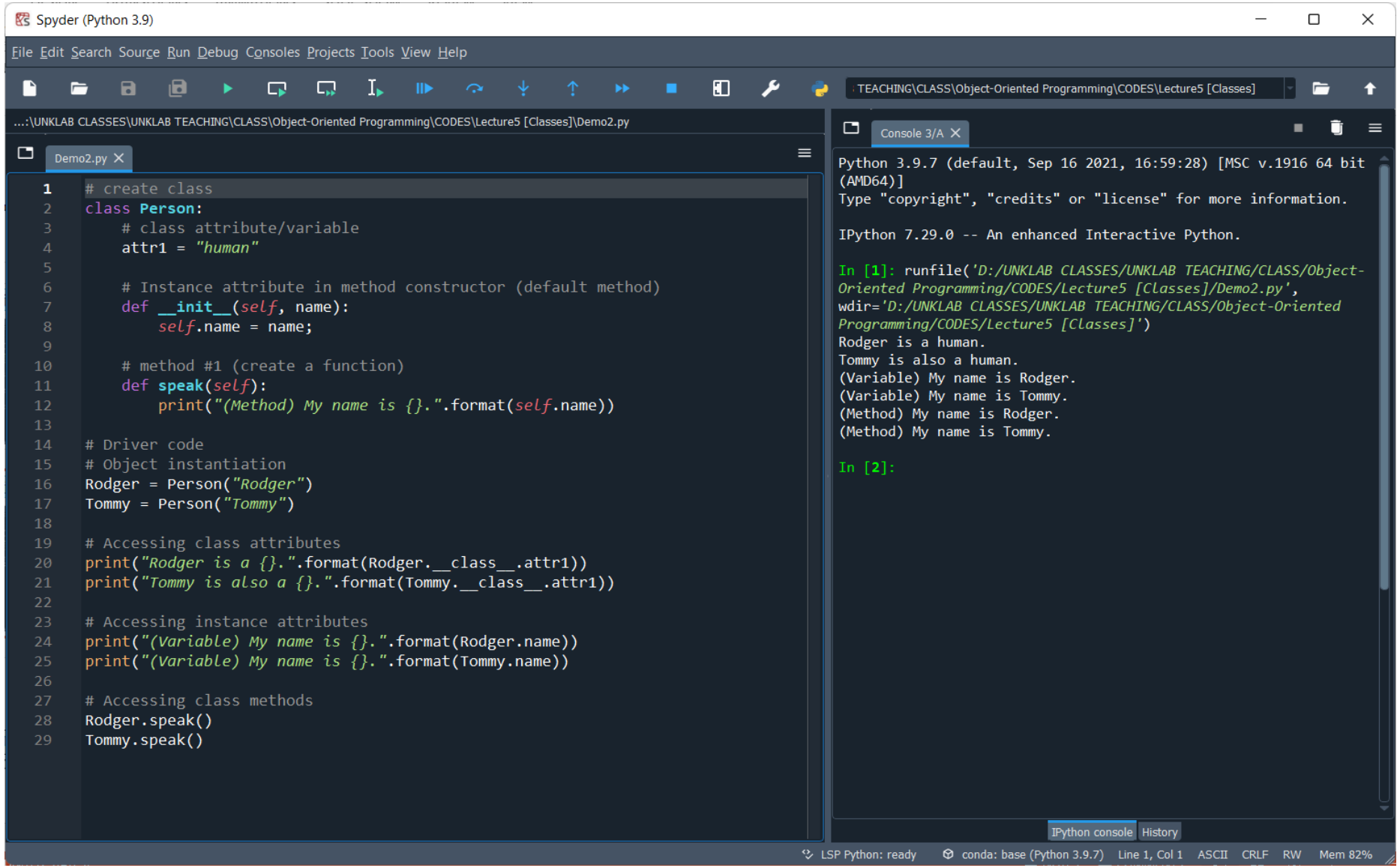
2. Tambahkan dua parameter pada *constructor*.

3. Gunakan *constructor* untuk menginisialisasi dua *variable* yang baru saja ditambahkan.

4. Buat dua method baru (*tampil_expire()* dan *tampil_negara_asal()*) dari class Produk. Method-method tersebut dapat mengembalikan nilai berupa String dan tanpa parameter.

5. Tampilkan output dari dua method tersebut menggunakan *object* dari *class*.

Exercise #2 (Class & Objects)



Modifikasi source code pada file Demo2.py:

1. Dalam **class Person**, tambahkan 4 *variable* (***age, address, phone_number*** dan ***gender***) dengan type String semua.

2. Tambahkan 4 parameter pada *constructor* untuk menginisialisasi empat *variable* yang baru saja ditambahkan tersebut.

3. Buat 4 method baru (*view_age(), view_address(), view_phone()* dan *view_gender()*) dari class Person. Method-method tersebut dapat mengembalikan nilai berupa String dan tanpa parameter.

4. Tampilkan output dari ke-empat method tersebut menggunakan object dari class.

Class Activity #1

1. Satu kelompok max 2 students.

2. Jelaskan (*tentang class, variable dan method*) dan buat report dari dua Python files pada exercise di atas.

3. Screenshot output program yang telah dibuat.

4. Submit per student (masing-masing di Google ClassRoom).

5. Submit report dalam format PDF.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T